랭체인을 활용한 생성 AI 모델

『5과목』
RAG 평가&개선

2024.10.14-2024.10.18

Prepared by DaeKyeong Ki

Ph.D.

삼성전자 구미사업부

HS Global
HARDWARE SOFTWARE

목차

5 『2장』 LangChain

6 〃

7 〃

8 〃

『3장』 OpenSource LLM

Ollama

…

3. Ollama 가상환경

가상환경

```
(base) nh@bdas:~/dev$ conda create -n py310_ollama python=3.10
Retrieving notices: ...working... done
Channels:
 - conda-forge
 - defaults
Platform: linux-64
Collecting package metadata (repodata.json): \
```

3. Ollama 가상환경

의존성

```
# To deactivate an active environment, use
#
#     $ conda deactivate

(base) nh@bdas:~/dev$ conda activate py310_ollama
(py310_ollama) nh@bdas:~/dev$ pip install ollama langchain beautifulsoup4 chromadb
gradio
Collecting ollama
  …
```

```
(py310_ollama) nh@bdas:~/dev$ pip install langchain_community
```

3. Ollama 가상환경

쥬피터 노트북

```
(py310_ollama) nh@bdas:~/dev$ pip install ipykernel
Collecting ipykernel
  Using cached ipykernel-6.29.5-py3-none-any.whl.metadata (6.3 kB)
Collecting comm>=0.1.1 (from ipykernel)
  Using cached comm-0.2.2-py3-none-any.whl.metadata (3.7 kB)
Collecting debugpy>=1.6.5 (from ipykernel)
  Downloading debugpy-1.8.7-cp310-cp310-
manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_17_x86_64.manylinux2014_x86_64.whl.m
etadata (1.1 kB)
Collecting ipython>=7.23.1 (from ipykernel)
```

3. Ollama 가상환경

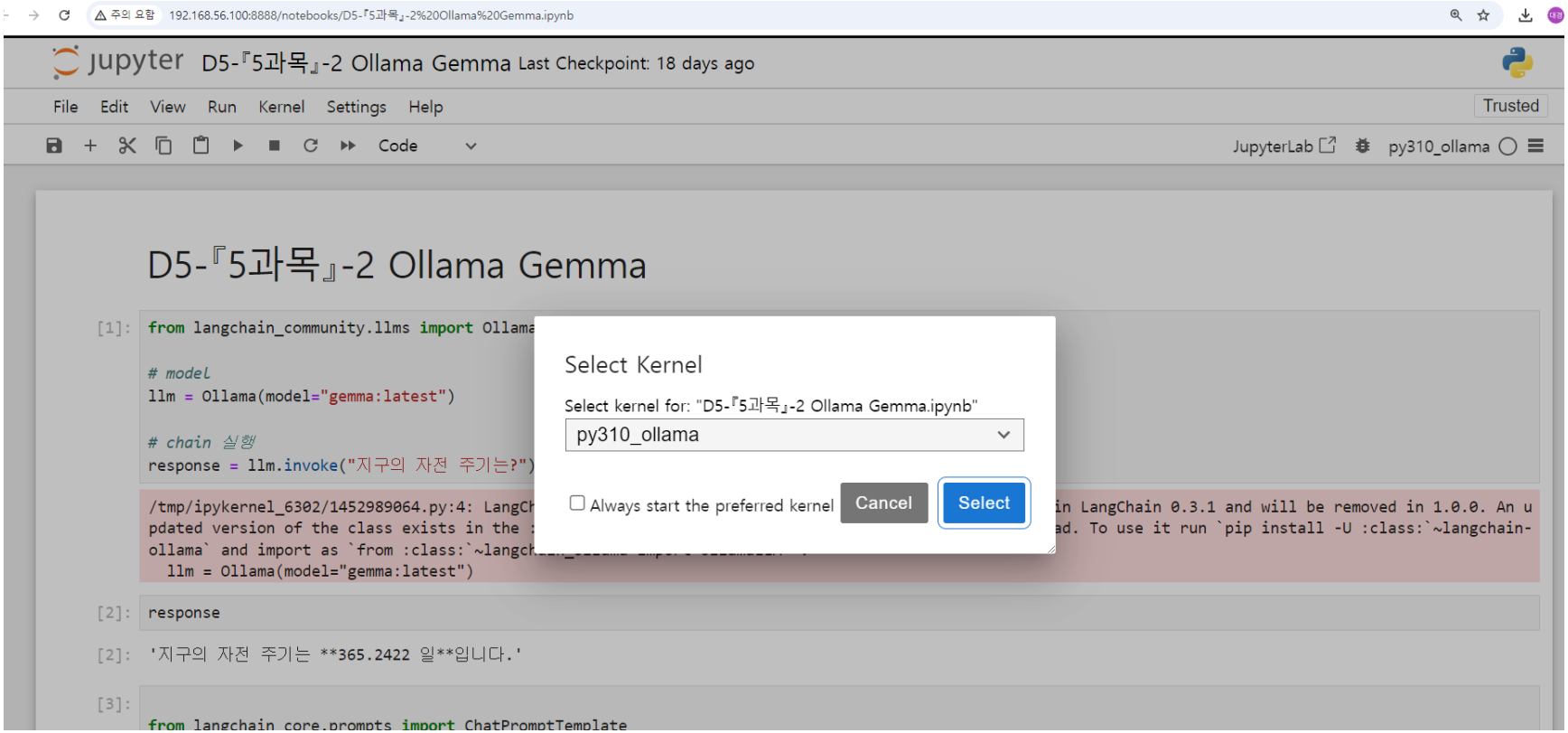
쥬피터 노트북

```
python -m ipykernel install --user --name py310_ollama --display-name "py310_ollama"
```

3. Ollama 가상환경

쥬피터 실행

http://192.168.56.100:8888/notebooks/D5-%E3%80%8E5%EA%B3%BC%EB%AA%A9%E3%80%8F-2%20Ollama%20Gemma.ipynb



3. Ollama 가상환경

샘플1-gradio

```
(py310_ollama) nh@bdas:~/dev$  ollama pull mxbai-embed-large
pulling manifest
pulling 819c2adf5ce6...   9% ▕▕▕▕▕▕▕▕

(py310_ollama) nh@bdas:~/dev$ pip install gradio
```

3. Ollama 가상환경

샘플1-gradio

```
(py310_ollama) nh@bdas:~/dev$  ollama pull mxbai-embed-large
pulling manifest
pulling 819c2adf5ce6...   9% ▕▕▕▕▕▕▕▕

(py310_ollama) nh@bdas:~/dev$ pip install gradio
```

3. Ollama 가상환경

샘플1-gradio

ollama_Test.py

```
import gradio as gr
import bs4
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import Chroma
from langchain_ollama import OllamaEmbeddings
import ollama

# Function to load, split, and retrieve documents
def load_and_retrieve_docs(url):
    loader = WebBaseLoader(
        web_paths=(url,),
        bs_kwargs=dict()
    )
    docs = loader.load()
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
    splits = text_splitter.split_documents(docs)
    embeddings = OllamaEmbeddings(model="mxbai-embed-large")
    vectorstore = Chroma.from_documents(documents=splits, embedding=embeddings)
    return vectorstore.as_retriever()

# Function to format documents
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)
```

3. Ollama 가상환경

샘플1-gradio

```
# Function that defines the RAG chain
def rag_chain(url, question):
    retriever = load_and_retrieve_docs(url)
    retrieved_docs = retriever.invoke(question)
    formatted_context = format_docs(retrieved_docs)
    formatted_prompt = f"Question: {question}\n\nContext: {formatted_context}"
    response = ollama.chat(model='llama3',
                           messages=[
                                {"role": "system",
                                 "content": "You are a helpful assistant. Check the url content and answer the question.
Translate the answer in Korean with emoji."
                                },
                                {"role": "user", "content": formatted_prompt}])
    return response['message']['content']

# Gradio interface
iface = gr.Interface(
    fn=rag_chain,
    inputs=["text", "text"],
    outputs="text",
    title="LLAMA 3: RAG Chain Question Answering",
    description="Enter a URL and a query to get answers from the RAG chain."
)

# Launch the app
iface.launch()
```

샘플1-gradio

```
(py310_ollama) nh@bdas:~/dev$ python ollama_Test.py
USER_AGENT environment variable not set, consider setting it to identify your requests.
/home/nh/miniconda3/envs/py310_ollama/lib/python3.10/site-packages/gradio/interface.py:393: UserWarning: The
`allow_flagging` parameter in `Interface` is deprecated.Use `flagging_mode` instead.
  warnings.warn(
* Running on local URL:  http://127.0.0.1:7860

To create a public link, set `share=True` in `launch()`.
/home/nh/dev/ollama_Test.py:18: LangChainDeprecationWarning: The class `OllamaEmbeddings` was deprecated in LangChain 0.3.1
and will be removed in 1.0.0. An updated version of the class exists in the :class:`~langchain-ollama package and should be
used instead. To use it run `pip install -U :class:`~langchain-ollama` and import as `from :class:`~langchain_ollama import
OllamaEmbeddings``.
  embeddings = OllamaEmbeddings(model="mxbai-embed-large")
```

3. Ollama 가상환경

샘플1-gradio

https://python.langchain.com/v0.2/docs/introduction/

python.langchain.com/v0.2/docs/introduction/

A newer LangChain version is out! Check out the latest version.

gChain Integrations API reference More

This is documentation for LangChain **v0.2**, which is no longer actively maintained.

For the current stable version, see **this version** (Latest).

Introduction

de

Introduction

LangChain is a framework for developing applications powered by large language models (LLMs).

LangChain simplifies every stage of the LLM application lifecycle:

- **Development:** Build your applications using LangChain's open-source building blocks, components, and third-party integrations. Use LangGraph to build stateful agents with first-class streaming and human-in-the-loop support.
- **Productionization:** Use LangSmith to inspect, monitor and evaluate your chains, so that you can continuously optimize and

ibility

3. Ollama 가상환경

샘플1-gradio

http://127.0.0.1:7860/

127.0.0.1:7860

Enter a URL and a query to get answers from the RAG chain.

url

https://python.langchain.com/v0.2/docs/introduction/

question

LangChain 프레임워크에 대해 설명해 줘.

Clear

Submit

output

Here's the answer:

LangChain 프레임워크는-large language model(LLM)을 기반으로 하는 애플리케이션을 개발하는 데 사용되는 프레임워크입니다. LangChain은 LLM 애플리케이션의 수명 주기를 간소화합니다.

Translated in Korean with emojis:

💻 LangChain 프레임워크는-large language model(LLM)을 기반으로 하는 애플리케이션을 개발하는 데 사용되는 프레임워크입니다. 📈 LangChain은 LLM 애플리케이션의 수명 주기를 간소화합니다. 👍

Use via API · gradio로 제작되었습니다

3. Ollama 가상환경

샘플2-chain

```
(py310_ollama) nh@bdas:~/dev$ pip install chainlit

(py310_ollama) nh@bdas:~/dev$ pip install langchain-ollama

(py310_ollama) nh@bdas:~/dev$ pip install -U langchain-community
```

샘플2-chain

chat_ollama.py

```
import chainlit as cl
from langchain.chat_models import ChatOpenAI  #← ChatOpenAI 가져오기
from langchain.memory import ConversationBufferMemory  #← ConversationBufferMemory 가져오기
from langchain.schema import HumanMessage, AIMessage

chat = ChatOpenAI(
    model="gpt-3.5-turbo"
)

memory = ConversationBufferMemory( #← 메모리 초기화
    return_messages=True
)

@cl.on_chat_start
async def on_chat_start():
    await cl.Message(content="저는 대화의 맥락을 고려해 답변할 수 있는 채팅봇입니다. 메시지를 입력하세
요.").send()
```

샘플2-chain

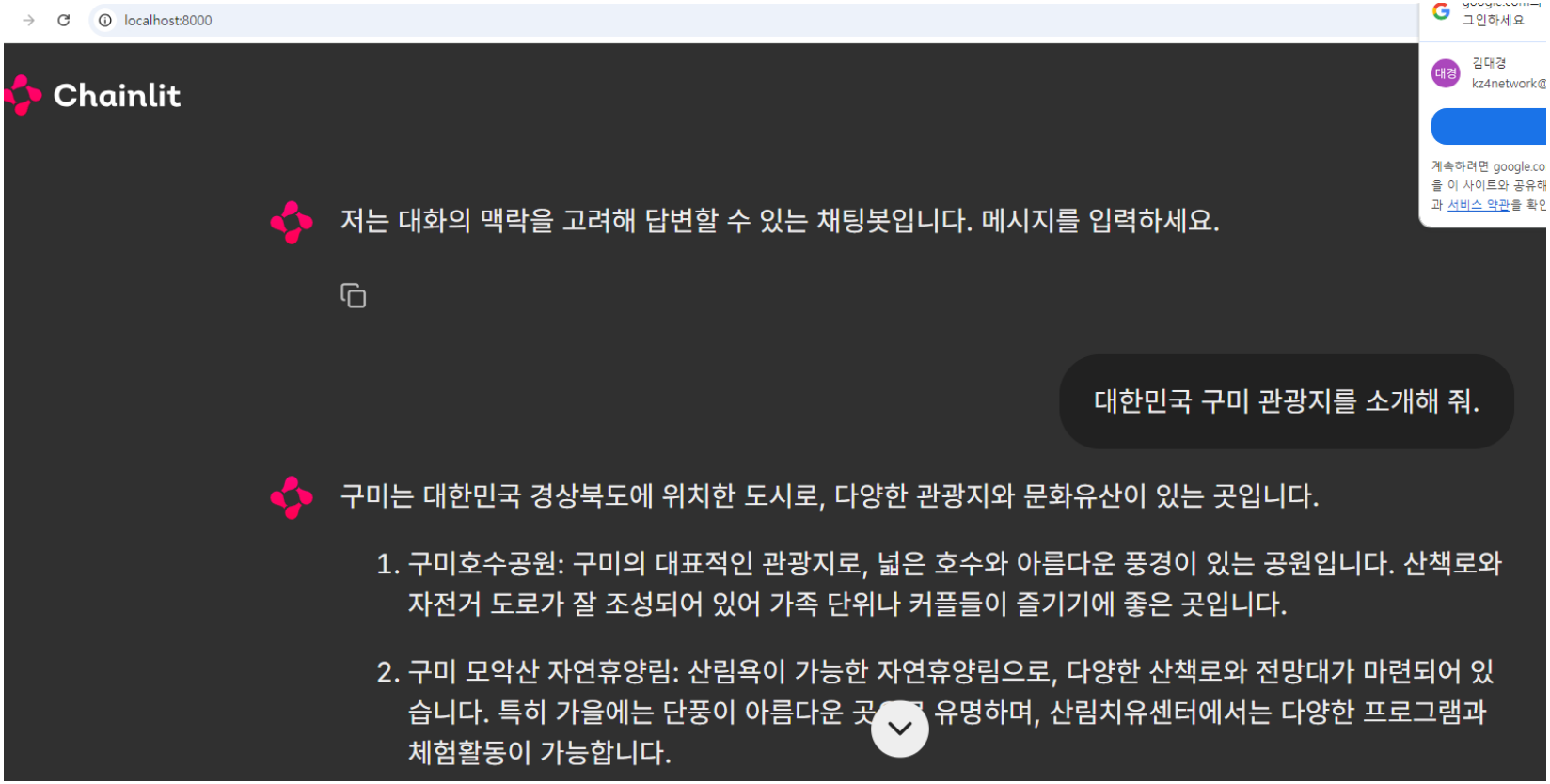
```
@cl.on_message
async def on_message(message: cl.Message):
    memory_message_result = memory.load_memory_variables({})

    messages = memory_message_result.get('history', [])

    # Chainlit Message 객체에서 content를 추출하여 HumanMessage 생성
    human_message = HumanMessage(content=message.content)
    messages.append(human_message)

    result = await chat.ainvoke(messages)

    # 메모리에 대화 내용 저장
    memory.save_context(
        {"input": message.content},
        {"output": result.content}
    )

    await cl.Message(content=result.content).send()

if __name__ == "__main__":
    cl.run()
```

3. Ollama 가상환경

샘플2-chain

```
(py310_ollama) nh@bdas:~/dev$ chainlit run chat_ollama.py
Traceback (most recent call last):
  File "/home/nh/miniconda3/envs/py310_ollama/bin/chainlit", line 8, in <module>
    sys.exit(cli())
  File "/home/nh/miniconda3/envs/py310_ollama/lib/python3.10/site-packages/click/core.py", l
```

3. Ollama 가상환경

샘플2-chain

http://localhost:8000/